We can see the entire padding sequence is used (note that it gets truncated if the padded string would exceed the desired length), and the padding is in both directions (`STR_PAD_LEFT` and `STR_PAD_RIGHT` are the other constants available).

For multi-byte data, character length (as Unicode codepoints) is of more importance than byte length. A brand new addition in PHP 8.3, `mb_str_pad()` has an almost identical signature to `str_pad()`, but it treats the second argument as the desired number of codepoints in the resulting string, rather than the number of bytes. This has an impact on Unicode multi-codepoint characters: for example, flags are composed of four codepoints:

```
$paddingChars = "\u{1F3F3}\u{FE0F}\u{200D}\u{1F308} \u{1F3F4}\u{200D}\
u{2620}\u{FE0F}";
$text = 'Text';

print $paddingChars . PHP_EOL;
print mb_str_pad($text, 13, $paddingChars, STR_PAD_LEFT);
```

```
Output:
🏳️‍🌈 🏴‍☠️
Text
```

We have two flag symbols in our padding string, but they're constructed from four codepoints each (plus an ASCII space for another codepoint), so we need a target length of *original string length* + 9. Look what happens if we accidentally truncate the padding string:

```
print mb_str_pad($text, 5, $paddingChars, STR_PAD_LEFT);
```

```
Output:
🏳
Text
```

We get a white flag as our padding character, because only the first codepoint made it into our padding: U+1F3F3 ("Waving White Flag"). Keep an eye on things when using the more exotic symbols like emojis. As before, this is a "bald man" multi-codepoint example; an ancient GUI terminal might garble the output.

mb_strimwidth()